

# Day 6: Withdraw Amount with Limit

## 1. Day Number

Day 6

## 2. Topic Name

### Conditions with Multiple Rules

Today you will learn how a program checks several conditions before allowing an action.

## 3. Connection with Yesterday

Yesterday, you deposited money into a bank account.

Today, you will withdraw money, but the withdrawal should happen only when all banking rules are satisfied.

The program must check:

- Is the withdrawal amount positive?
  - Is enough balance available?
  - Is the amount within the withdrawal limit?
- 

## 4. Important Topics

- if-elif-else
  - Comparison operators
  - Logical validation
  - Multiple conditions
  - Functions
  - User input
  - Type conversion
- 

## 5. Foundational Notes

### What is a condition?

A condition is a question whose result is either:

- True
- False

Example:

```
age >= 18
```

This condition checks whether the age is at least 18.

## Comparison operators

| Operator | Meaning                  | Example           |
|----------|--------------------------|-------------------|
| >        | Greater than             | amount > 0        |
| <        | Less than                | amount < limit    |
| >=       | Greater than or equal to | balance >= amount |
| <=       | Less than or equal to    | amount <= limit   |
| ==       | Equal to                 | amount == 0       |
| !=       | Not equal to             | amount != 0       |

## What is if-elif-else?

Use if-elif-else when the program needs to check multiple possibilities.

```
if first_condition:
    # Run when the first condition is true
elif second_condition:
    # Run when the second condition is true
else:
    # Run when no previous condition is true
```

Python checks the conditions from top to bottom. Once it finds a true condition, it runs that block and skips the remaining blocks.

## Why validation order matters

Suppose the user enters 0.

Before checking the balance or withdrawal limit, the program should first check whether the amount is positive.

A useful validation order is:

1. Check whether the amount is positive.
2. Check whether it exceeds the balance.
3. Check whether it exceeds the withdrawal limit.
4. Otherwise, allow the withdrawal.

---

## 6. Easy Example

Imagine an ATM has these values:

```
Balance = $1,000
Withdrawal limit = $500
Requested amount = $300
```

Check the rules:

```
Is $300 positive?           Yes
Is $300 greater than $1,000? No
Is $300 greater than $500?  No
```

All checks are passed, so the withdrawal can be allowed.

The new balance would be:

```
$1,000 - $300 = $700
```

Another example:

```
Balance = $1,000  
Withdrawal limit = $500  
Requested amount = $600
```

The account has enough balance, but \$600 is greater than the \$500 withdrawal limit. Therefore, the withdrawal should be rejected.

---

## 7. Problem Statement

Create a program that:

1. Stores an account balance.
  2. Stores a withdrawal limit.
  3. Asks the user to enter a withdrawal amount.
  4. Allows the withdrawal only when:
    - The amount is greater than 0.
    - The amount is not greater than the available balance.
    - The amount is not greater than the withdrawal limit.
  5. Displays an appropriate error message when any rule fails.
  6. Displays the remaining balance after a successful withdrawal.
- 

## 8. Concepts Used

### Variables

Variables store the balance, limit, and withdrawal amount.

```
balance  
withdrawal_limit  
withdrawal_amount
```

### Input

`input()` receives the withdrawal amount from the user.

Remember that `input()` returns a string.

### Type conversion

Convert the entered amount into a number before comparing it.

For example, you may use:

```
float()
```

This allows values such as:

```
250
250.50
```

## Conditions

Use conditions to validate the amount before subtracting it from the balance.

## Function

A function keeps the withdrawal logic separate and organized.

A possible function responsibility is:

```
Receive balance, limit, and requested amount
Validate the requested amount
Display the result
Return the updated balance
```

---

## 9. Thought Process

Follow this reasoning before writing the program:

### Step 1: Store the account information

Create variables for:

- Current balance
- Maximum withdrawal limit

### Step 2: Ask for the withdrawal amount

Take input from the user and convert it into a number.

### Step 3: Check whether the amount is valid

The amount must be greater than 0.

If the amount is 0 or negative, show an invalid amount message.

### Step 4: Check the available balance

If the requested amount is greater than the balance, the withdrawal cannot happen.

### Step 5: Check the withdrawal limit

Even when enough balance is available, the requested amount must not exceed the limit.

### Step 6: Complete the withdrawal

When all validations pass:

```
new balance = current balance - withdrawal amount
```

Then display:

- Withdrawal successful
- Withdrawn amount
- Remaining balance

---

## 10. Pseudocode

```
START

SET balance to a fixed amount
SET withdrawal_limit to a fixed amount

ASK user for withdrawal amount
CONVERT withdrawal amount to a number

DEFINE a withdrawal function

    IF withdrawal amount is less than or equal to zero
        DISPLAY "Amount must be greater than zero"

    ELSE IF withdrawal amount is greater than balance
        DISPLAY "Insufficient balance"

    ELSE IF withdrawal amount is greater than withdrawal limit
        DISPLAY "Withdrawal limit exceeded"

    ELSE
        SUBTRACT withdrawal amount from balance
        DISPLAY "Withdrawal successful"
        DISPLAY remaining balance

    RETURN balance

CALL the withdrawal function

END
```

---

## 11. Suggested Solving Approach: Functional Approach

Create a function similar to:

```
def withdraw_amount(balance, withdrawal_limit, amount):
```

The function should:

1. Receive the current balance.
2. Receive the withdrawal limit.
3. Receive the requested amount.
4. Perform all validations.
5. Subtract the amount only after every validation passes.
6. Return the updated balance.

Outside the function:

1. Store the initial balance.
2. Store the withdrawal limit.

3. Ask the user for an amount.
4. Call the function.

This structure separates the program into two parts:

```
Input section
  ↓
Withdrawal function
  ↓
Result
```

## 12. Easy Edge Cases

### Case 1: Amount greater than balance

```
Balance: $1,000
Limit: $1,500
Amount: $1,200
```

Expected result:

```
Insufficient balance
```

### Case 2: Amount greater than withdrawal limit

```
Balance: $2,000
Limit: $500
Amount: $700
```

Expected result:

```
Withdrawal limit exceeded
```

### Case 3: Amount is 0

```
Amount: $0
```

Expected result:

```
Amount must be greater than zero
```

### Case 4: Amount is negative

```
Amount: -$100
```

Expected result:

```
Amount must be greater than zero
```

### Case 5: Amount equals the balance

```
Balance: $500
Limit: $500
Amount: $500
```

The withdrawal should be allowed because the amount is not greater than the balance or limit.

Remaining balance:

```
$0
```

### Case 6: Amount equals the limit

```
Balance: $1,000
Limit: $500
Amount: $500
```

The withdrawal should be allowed because the amount does not exceed the limit.

---

## 13. Expected Input

Example 1:

```
Enter withdrawal amount: 300
```

Example 2:

```
Enter withdrawal amount: 700
```

Example 3:

```
Enter withdrawal amount: 0
```

---

## 14. Expected Output

### Successful withdrawal

```
Withdrawal successful.
Withdrawn amount: $300.00
Remaining balance: $700.00
```

### Insufficient balance

```
Withdrawal failed.
Insufficient balance.
```

### Limit exceeded

```
Withdrawal failed.  
The amount exceeds the withdrawal limit.
```

### Invalid amount

```
Withdrawal failed.  
The amount must be greater than zero.
```

---

## 15. Hint Only

Create a function that accepts `balance`, `withdrawal_limit`, and `amount`.

Check the invalid cases first using `if` and `elif`. Keep the successful withdrawal inside the final `else` block.

The important calculation is:

```
updated balance = balance - amount
```

Do not subtract the amount until all three validation rules have passed.